C 言語基礎編④　テキスト

2021 年 9 月版

明治大学
生田メディア支援事務室

目次

1. ポインタ 3
1.1. 変数のアドレス 3
1.2. ポインタと変数 5
1.3. ポインタと関数 7
1.4. ポインタと配列 10
1.5. 演習問題 12
2. 構造体 15
2.1. 構造体の宣言 16
2.2. 構造体型変数の宣言 16
2.3. typedef を用いた宣言 18
2.4. 演習問題 19
3. 参考文献 23
4. 演習問題解答例 24
4.1. 1 章演習問題 24
4.2. 2 章演習問題 27

1. ポインタ

C 言語基礎編①②講習会では、変数の値がコンピュータのメモリに記憶されていることを学びました。メモリは 1 列に並べられたロッカーのようなイメージで、その 1 つ 1 つにロッカーの場所を示す番号（アドレス）が割り振られています。変数を宣言し、アドレスに変数名を付けることで、私たちは普段アドレスの存在を意識しなくても、変数に操作を加えることができています。

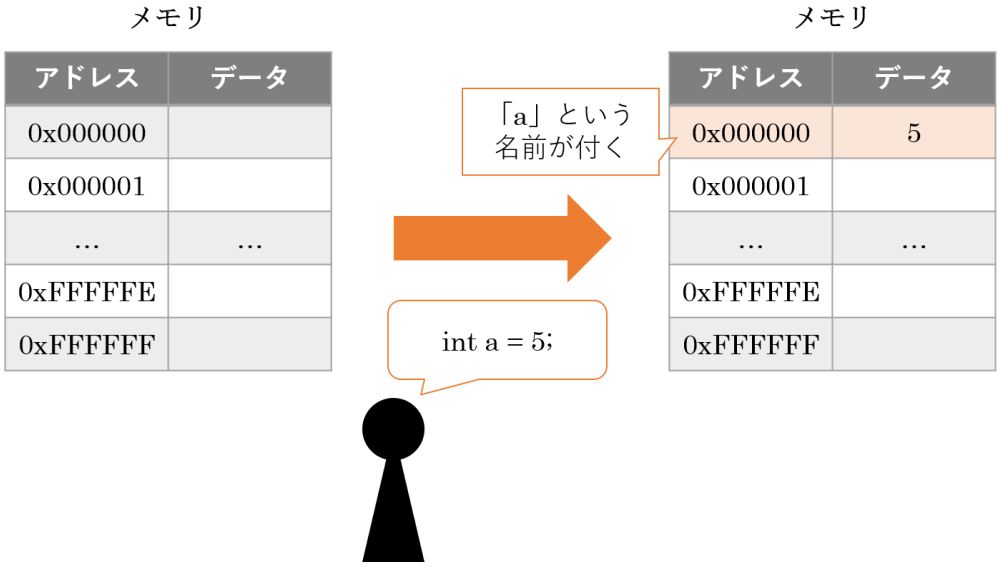

図1.　メモリの構造と変数宣言

しかし、C 言語では、この「アドレス」を直接操作する必要がある事態がしばしば発生します。そこで本章では、「ポインタ」というアドレスを格納する特殊な変数について学習していきます。

1.1.　変数のアドレス

まずは、実際に変数のアドレスを確認してみましょう。ある変数のアドレスを知りたいとき、以下のように記述します。

```
&変数名
```

変数名の前にある「&」は「アドレス演算子」という、変数のアドレスを示したい場合に用いる演算子です。アドレス演算子を変数の前につけることで、その変数のアドレスを知ることができます。

次に、アドレス演算子を用いて、実際にアドレスを出力してみましょう。

＜sample1.c＞

```
#include <stdio.h>

int main(void)
{
        int a = 5;
        printf( “変数 a の値は%d です¥n”, a);
        printf( “変数 a のアドレスは%p です¥n”, &a);

        return 0;
}
```

＜実行結果＞

```
変数 a の値は 5 です
変数 a のアドレスは 0x000000 です
```

このとき結果で出力された「0x000000」という値が、変数「a」のアドレスであり、「&a」と記述することで確認ができます（この値はあくまで例です。実際の値は実行環境によって変化します。）。

また、アドレスの値を出力するときは、変換指定子に「%p」を用います。

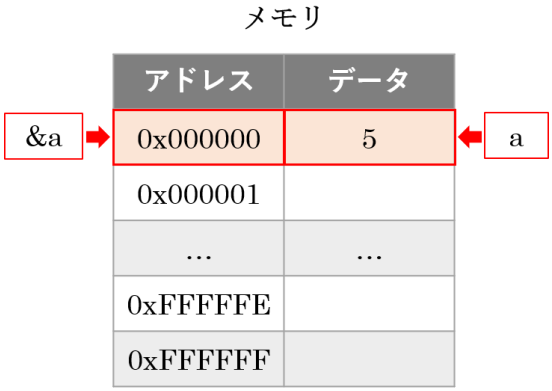

図2.　変数の値とアドレス

1.2.　ポインタと変数

前節では、ある変数のアドレスを知るために、変数名に&記号を付けて確認していました。このアドレスを格納することができるのが、「ポインタ」と呼ばれる特殊な変数です。

ポインタの宣言は、以下のように行います。

```
型名　*ポインタ名；
```

このように宣言することで、「ある型の変数のアドレスを格納することができるポインタ」を定義することができます。変数名の前に「*」を付けることで、ポインタの宣言であることを表しています。

＜sample2.c＞

```
#include <stdio.h>

int main(void)
{
        int a;  //変数 a の宣言
        int *pa;  //ポインタ pa の宣言

        a = 5;
        pa = &a;  //pa に変数 a のアドレスを格納

        printf( “変数 a の値は%d です¥n”, a);
        printf( “変数 a のアドレスは%p です¥n”, &a);
        printf( “ポインタ pa の値は%p です¥n”, pa);

        return 0;
}
```

＜実行結果＞

```
変数 a の値は 5 です
変数 a のアドレスは 0x000000 です
ポインタ pa の値は 0x000000 です　//ポインタ pa の値は a のアドレスと同じ
```

「a」という変数のアドレスを「pa」というポインタに格納することで、a のアドレスを通じて変数 a とポインタ pa にはある「関係」が生まれています。この関係のことを「pa は変数 a をさす」といい、「pa の値が変数 a の場所をさし示すようになった」とも言い換えられます。

逆に、ポインタがさし示す変数の値を知ることもできます。ポインタから変数の値をたどる際には、ポインタに対して、「*」という演算子（「間接参照演算子」という）を用います。

```
*ポインタ名；　//ポインタがさし示す変数の値を参照
```

実際のソースプログラムで使用してみましょう。

＜sample3.c＞

```
#include <stdio.h>

int main(void)
{
        int a = 5;
        int *pa = &a;　//宣言と同時にアドレスを格納することもできる

        printf( “変数 a の値は%d です¥n”, a);
        printf( “ポインタ pa がさす変数の値は%d です¥n”, *pa);
        //間接参照演算子を用いてポインタがさし示すアドレスに格納されている値を出力
        printf( “↑の値に 5 を加えると%d です¥n”, (*pa) += 5);
        //ポインタから値を直接変更することも可能

        return 0;
}
```

＜実行結果＞

```
変数 a の値は 5 です
ポインタ pa がさす変数の値は 5 です
↑の値に 5 を加えると 10 です
```

今、アドレスを通じて変数 a とポインタ pa は繋がっているため、アドレスをもとに実際に格納されている変数の値を参照することもできるのです。

図3.　ポインタと変数

ここまで学習した内容を、一度整理します。これまで扱ってきた変数と新しく出てきたポインタでは、それぞれどんな値を参照したいかによって、扱い方が変わります。表 1 を参考にしてください。

表 1.　ポインタと変数の区別

種類	宣言	値の参照	アドレスの参照
変　数	型名　変数名；	変数名	&変数名
ポインタ	型名　*ポインタ名；	*ポインタ名	ポインタ名

1.3.　ポインタと関数

ポインタの非常に重要な役割の 1 つとして、関数の引数としての使用方法があります。

まず、以下のソースプログラムを確認してみましょう。2 つの引数を取り、その値を入れ替える swap という関数を定義しています。

＜sample4.c＞

```
#include <stdio.h>

void swap ( int num1, int num2){
        int tmp = num1;
        num1 = num2;
        num2 = tmp;
}

int main(void)
{
        int x = 5, y = 10;
        printf( “x=%d, y=%d を入れ替えると¥n”, x, y);

        swap (x, y);
        printf( “x=%d, y=%d です¥n”, x, y);

        return 0;
}
```

関数 swap では、片方の値を一時的に tmp に格納することで、値の入れ替えを行っています。

main 関数では、x=5, y=10 をそれぞれ引数として指定し、関数 swap を呼び出しています。実際にこのプログラムを実行してみましょう。

＜実行結果＞

```
x=5, y=10 を入れ替えると
x=5, y=10 です
```

結果はこのようになり、値の入れ替えが行われていません。

ここで、関数における「引数」の性質を思い出してみましょう[1]。関数が呼び出されるとき、関数は引数に指定された変数の値を受け取り、関数内での処理に用いています。ここで関数が扱えるのは、あくまで変数の「値」であり、「変数そのもの」に操作を加えることはできません。このような関数への引数の引き渡しの方式を、「値渡し」と呼びます。

関数 swap は、引数である x と y の値そのものに変更を加えようとしていますが、受け取っているのはあくまで 2 つの引数の値である 5 と 10 という数値のみなので、そのような変更はできません。

このような場面で、非常に便利な働きをしてくれるのがポインタです。ポインタは、ある変数のアドレスを参照するだけでなく、その変数が格納している値にも変更を加えることが可能でした[2]。つまり、関数の引数に、変数ではなくポインタを渡すことで、関数内で引数そのものにも変更を加えることが可能になります。

以下に、先ほどの sample4.c に変更を加えたものを示します。

＜sample5.c＞

```
#include <stdio.h>

void swap ( int *num1, int *num2){   //引数にポインタを指定している
        int tmp = *num1;
        *num1 = *num2;
        *num2 = tmp;   //アドレスではなく値を参照するため、「*」演算子を付ける
}

int main(void)
{
        int x = 5, y = 10;
        printf( “x=%d, y=%d を入れ替えると¥n”, x, y);

        swap ( &x, &y);   //関数呼び出しの際には、値ではなくアドレスを渡す
        printf( “x=%d, y=%d です¥n”, x, y);

        return 0;
}
```

[1] C 言語基礎編③テキスト 13, 14 ページ　参照
[2] 本テキスト 6 ページ sample3.c　参照

＜実行結果＞

```
x=5, y=10 を入れ替えると
x=10, y=5 です
```

sample5.c では、関数 swap に値ではなくアドレスを渡すことで、そのアドレスに格納されている値を直接扱うことができるようになりました。

このような引数の引き渡しの方式を、「参照渡し」といいます。より柔軟で効率的なプログラムを組む際に、非常に重要な概念となりますので、値渡しと参照渡しの違いは必ず理解しておきましょう。

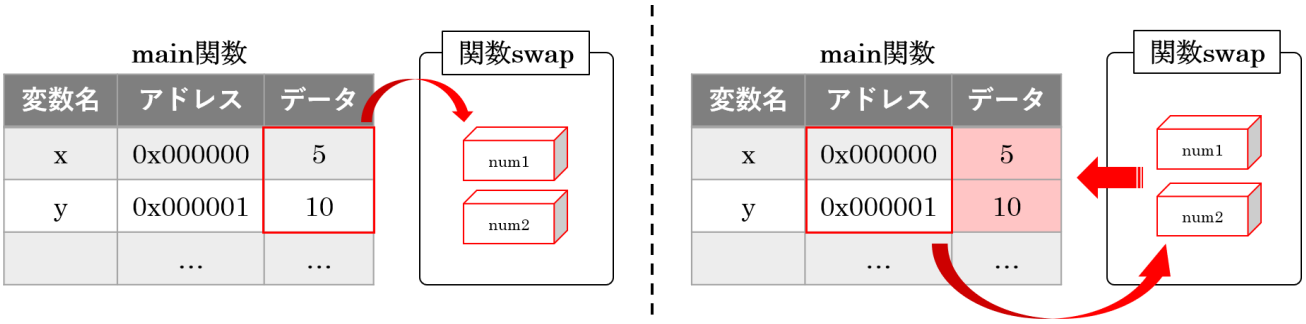

図4.　値渡し（左）と参照渡し（右）のイメージ

また、関数内の処理が複雑になると、引数で渡しているものの、更新をしてはならない変数を、誤って更新してしまうようなミスが起きかねません。そのような事態を防ぐために、仮引数に「const」という指定を付ける方法があります。

＜sample6.c＞

```
#include <stdio.h>

void plus10( int *x, const int *y){   //const を指定すると y は読み取り専用の変数として認識される
        *x += 10;
        *y += 10;   //関数定義の際にここでエラーが発生する
}
```

1.4.　ポインタと配列

ここまで、ポインタと変数や、ポインタと関数などの関係について学習してきました。しかし、扱う変数の数が増えれば増えるほど、配列の役割が重要になってきます。本節では、ポインタと配列の関係について学んでいきます。

まず、配列の各要素のアドレスを確認していきましょう。変数の場合と同様、先頭に「&」を付けるだけで、アドレスの確認ができます。

```
&配列名［要素番号］;
```

＜sample7.c＞

```
#include <stdio.h>

int main(void)
{
        int test[5], i;
        for ( i = 0; i < 5; i++){
                printf( “test[%d]のアドレスは%p です¥n”, i, &test[i]);
        }

        return 0;
}
```

＜実行結果＞

```
test[0]のアドレスは 0x0080 です
test[1]のアドレスは 0x0084 です
test[2]のアドレスは 0x0088 です
test[3]のアドレスは 0x008c です
test[4]のアドレスは 0x0090 です
```

さらに配列では、単に「配列名」を記述すると、配列の先頭要素のアドレスをさし示すことができます。

```
配列名 ;    //配列の先頭要素のアドレスを取得
*配列名 ;    //間接参照演算子を付ければ配列の先頭要素の値も取得可能
```

＜sample8.c＞

```
#include <stdio.h>

int main(void)
{
        int test[5] = { 10, 20, 30, 40, 50};
        printf( “test[0]の値は%d，アドレスは%p です¥n”, test[0], &test[0]);
        printf( “test の値は%d，アドレスは%p です¥n”, *test, test);

        return 0;
}
```

＜実行結果＞

```
test[0]の値は 10，アドレスは 0x0080 です
test の値は 10，アドレスは 0x0080 です
```

ここで、関数の引数に配列を指定するときのことを思い出してみましょう[3]。関数を呼び出す際、以下のように、引数には「配列名」のみを記述していました。

```
int main(void)   //メイン文
{
        ・・・
        関数名（配列名，・・・）；  //関数呼び出し　引数の指定は「配列名」のみでよい
        ・・・
}
```

これはつまり、「関数には配列の先頭要素のアドレスを渡している」ということになります。関数定義の際には、引数を「配列名［］」とし、この関数が配列を受け取るということを指定します。関数呼び出しの際には、引数として先頭要素のアドレスのみを受け取り、そこから他の要素のアドレスもたどることで、配列全体を扱うことができるのです。したがって、配列は「参照渡し」で関数に渡されることとなります。

[3] C 言語基礎編③テキスト 14 ページ　参照

1.5.　演習問題

【1】

以下のプログラム「pointer_test.c」を実行すると、実行結果がどのようになるか答えてみましょう。（コンパイル時に警告文が出ますが、実行には影響がないため気にしないでください）

＜pointer_var.c＞

```
#include <stdio.h>

int main(void)
{
	int num = 10;
	int *ptr_num = &num;

	if( num == ptr_num){
		printf( “num = ptr_num¥n”);
	}
	if( num == *ptr_num){
		printf( “num = *ptr_num¥n”);
	}
	if( &num == ptr_num){
		printf( “&num = ptr_num¥n”);
	}
	if( &num == *ptr_num){
		printf( “&num = *ptr_num¥n”);
	}

	return 0;
}
```

＜実行結果＞

【2】

以下の条件を満たすプログラム「pointer_func.c」を作成するために、網掛け部分を埋めてみましょう。

・2 つの引数を取り、それぞれの引数に対して以下のような操作を加える関数 add_times を作成する

i） 1 つ目の引数に、2 つ目の引数の値を加算して返す

ii） 2 つ目の引数に、1 つ目の引数の値を乗算して返す

・キーボードから任意の 2 つの値を入力し、関数 add_times の実引数に指定する

・関数 add_times の実行結果を画面に表示する

＜pointer_func.c＞

```
#include <stdio.h>

    add_times(                    ){

}

int main(void){

}
```

＜実行結果イメージ＞

```
入力値１:5
入力値２:3
入力値１+入力値２=8
入力値１*入力値２=15
```

【3】

以下の条件を満たすプログラム「pointer_list.c」を作成するために、網掛け部分を埋めてみましょう。

・引数として整数の配列を受け取り、要素を値の大きい順に入れ替える関数 sort_list を作成する

・main 関数で、要素の入れ替え前と入れ替え後の配列の値をそれぞれ表示する

＜pointer_list.c＞

```
#include <stdio.h>

    sort_list(                    ){

}

int main(void)
{

}
```

＜実行結果イメージ＞

```
23 45 11 89 32
 ↓降順ソート
89 45 32 23 11
```

2. 構造体

ここまで変数や配列、ポインタなど、C 言語の様々な便利な機能について学習してきました。例えば、配列では、同じ型の変数をひとまとめにすることで、処理を効率的に行うことができます。

では、異なる型のデータをひとまとめに扱いたいときにはどうすればいいでしょう。

例えば、あるクラスの生徒の情報をまとめたいとき、「名前」や「住所」は文字列型ですが、「出席番号」や「テストの点数」などは数値型で表されます。さらに言えば、同じ「点数」でも、科目ごとの点数は整数型ですが、平均点などは実数型となります。

このような複数の型のデータをひとまとまりに扱うことができれば、さらに便利に C 言語を使いこなすことができるでしょう。

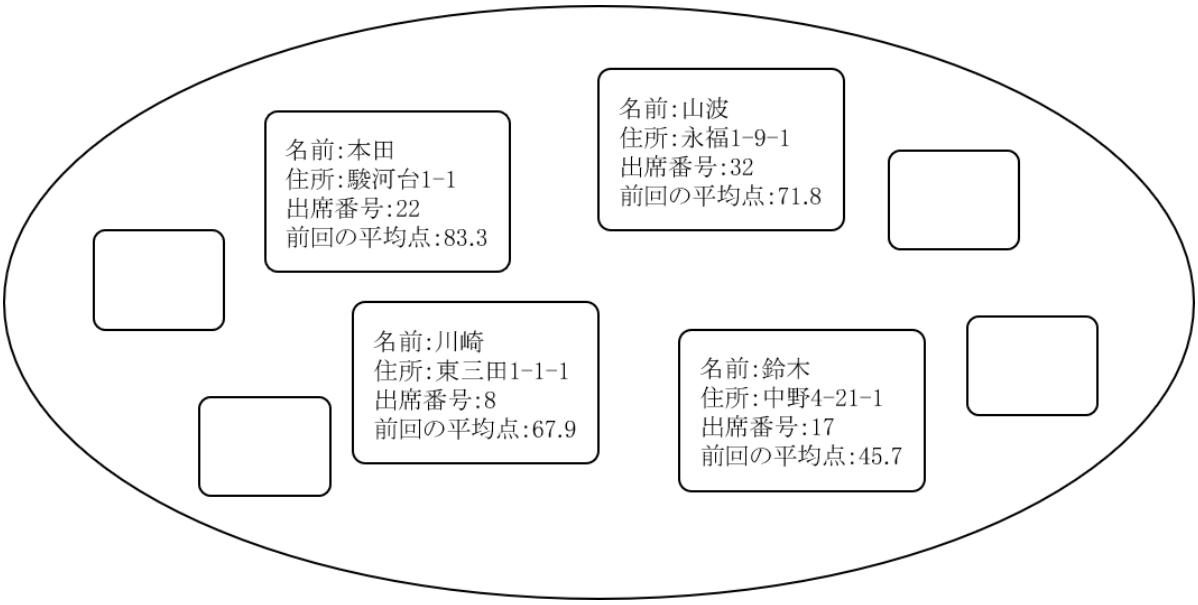

図5.　クラスメートの情報

本章では「異なる型の値をまとめて新しい型とする」ことのできる、構造体（structure）という C 言語の機能について学習していきたいと思います。

2.1.　構造体の宣言

構造体は、メンバと呼ばれる互いに関連のある 2 つ以上の変数で構成されます。

配列がどの要素も同じ型を持つのに対して、構造体はそれぞれのメンバが異なる型を持つことができます。

構造体を定義する一般的な形式を以下に示します。

＜構造体の宣言＞

```
struct タグ名 {
        型　メンバ 1 ;
        型　メンバ 2 ;
        型　メンバ 3 ;
        ・・・
        型　メンバ N ;
} (変数名);
```

例えば、図 5 のような生徒の情報をまとめる場合、

```
struct StudentRecord {
        char name[40];
        char address[100];
        int number;
        float score;
} list;
```

のようになります。「struct」を付けることで「StudentRecord」という新しい構造体型を、また最後に StudentRecord 型の変数「list」を宣言しています。

ここで、「name」「address」「number」「score」がそれぞれメンバとなっており、これらの変数を組み合わせたものが「struct StudentRecord」という構造体型です。

2.2.　構造体型変数の宣言

前節の例では、型の宣言と変数の宣言を同時に行っていますが、通常はこの 2 つの宣言を別々に行います。

＜構造体型変数の宣言＞

```
構造体型名　構造体変数名 ;
```

例）

```
struct StudentRecord {
        char name[40];
        char address[100];
        int number;
        float score;
};  //「StudentRecord」という構造体型を宣言

struct StudentRecord list;  //StudentRecord 型の変数「list」を宣言
```

また、構造体の各メンバにアクセスするには、変数名にピリオド（「.」）をつけ、そのあとにメンバ名を記述します。このピリオドを「ドット演算子」と呼びます。

＜メンバへのアクセス＞

```
構造体変数名. メンバ
```

例）

```
list.name = “本田”;
list.address = “駿河台 1-1”;
list.number = 22;
list.score = 83.3;
//「list」の各メンバに値を代入

StudentRecord list2 = { “川崎”, “東三田 1-1-1”, 8, 67.9};
//各メンバを一括で初期化することも可能
```

その他のメンバへのアクセス方法は、一般の変数と同じです。

```
scanf( “%d¥n”, &list.number);  //入力
int i;
for( i = 0; list.name[i]; i++){  //for 文でのアクセス
        printf( “%c”, list.name[i]);  //出力
}
```

構造体型の配列も宣言することができます。

＜構造体型配列の宣言＞

```
構造体型名　構造体型配列名［配列の要素数］;
```

例）

```
struct StudentRecord list[100];
```

例のように書くと、100 人分の生徒の情報を扱うことができるようになります。

配列の各要素へのアクセスも、一般の配列と同様に要素番号を指定して行います。

＜構造体型配列へのアクセス＞

```
list[0].name = “本田”;
list[1].name = “川崎”;
list[2].name = “山波”;
list[3].name = “鈴木”;
・・・
```

2.3. typedef を用いた宣言

「typedef」というキーワードを用いることで、自身が宣言した構造体に型名を付けることができます。構造体型の場合「struct ○○」という長い名前になってしまいますが、typedef を用いることで、これを省略して短い型名を設定することができます。

＜typedef の構文＞

```
typedef 型名　識別子；
```

このように宣言することで、元々の「型名」に「識別子」という名前を付けて用いることができます。以下に具体例を示します。

例）

```
typedef struct StudentRecord{
        ・・・
} Student;
```

このように記述することで、「struct StudentRecord」という構造体型に「Student」という新たな型名を付けることができます。

```
Student list3 = { “山波”, “永福 1-9-1”, 32, 71.8};
//struct StudentRecord 型の変数を宣言することと同じ
printf( “%s の出席番号は%d¥n”, list3.name, list3.number);

//実行結果：山波の出席番号は 32
```

2.4.　演習問題

※本演習問題は講習会内では取り扱いません。別途解答例を記載しますので、余力のある方は取り組んでみてください。

【1】

以下に示すような、「番号」、「氏名」、「4 回分のテストの点数」をメンバに持つ構造体を作成し、それぞれの値を表示するプログラム str_studentlist.c の網掛け部分を埋めてみましょう。

＜実行結果イメージ＞

番号	氏名	第1回	第2回	第3回	第4回
1	佐藤	88	76	69	79
2	鈴木	100	6	65	72
3	高橋	5	100	19	99
4	田中	98	98	96	92
5	伊藤	94	76	75	80

＜str_studentlist.c＞

```
#include <stdio.h>

#define N_TEST 4
#define N_HUMAN 5    //注釈 1

typedef struct GradeRecord{      //Record 型の構造体の定義
        int        ;
        char       ;
        int        ;
}Record;

int main(void)
{
        Record data[N_HUMAN] = {      //生徒の情報を Record 型変数 data に格納
                { 1, “佐藤”, {88, 76, 69, 79}},
                { 2, “鈴木”, {100, 6, 65, 72}},
                { 3, “高橋”, {5, 100, 19, 99}},
                { 4, “田中”, {98, 98, 96, 92}},
                { 5, “伊藤”, {94, 76, 75, 80}}
        };
```

```
        //一番上の行を表示
        printf( “番号　氏名　　”);
        int i, j;
        for( i = 0; i < N_TEST; i++){
                printf( “第%d 回 ”, i+1);
        }
        printf( “¥n”);
        //生徒の情報を表示
        for( i = 0; i <          ; i++){
                printf( “%3d    ”,          );
                printf( “%-4s   ”,          );
                for( j = 0; j <          ; j++){
                        printf( “%6d ”,          );
                }
                printf( “¥n”);
        }

        return 0;
}
```

※注釈 1：関数の外側に「#define 定数名 数値」と記述することで、数値を文字列に置き換えることができます。例えば「str_studentlist.c」では「N_TEST」という文字列が「4」という数値に置き換えられてプログラムが実行されます。

#define を用いると便利な場面として、配列の宣言があります。配列を宣言する際、要素数の指定には変数を用いることはできません。しかし、#define で定義した文字列は定数となるので、要素数の指定に用いることができます。このような定義の方式を「マクロ定義」と呼びます。

```
#define N 3   //N を 3 と定義する（マクロ定義）
int n = 3;   //int 型の変数 n に 3 を代入（変数の定義）

int array[n];   //変数を配列の要素数に指定することはできないため、コンパイルエラーが発生する
int array[N];   //N は定数として認識されるためコンパイルが可能になる
```

#define の見かけ上の動作は、typedef と非常によく似ています。しかし、コンピュータ内での処理は全く異なりますので注意しましょう。構造体の型を定義したいときには typedef を、何かの数値を定数として扱いたいときには、#define を用います。typedef と#define の処理の違いについて、参考となる URL を記載しますので、興味がある方は一読してみてください。

https://hotnews8.net/programming/tricky-code/language-specification03

（【C 言語】typedef と#define の違い）

【2】

【1】で作成した Record 型の配列 data から、各生徒のテストの合計点を求め、その生徒の名前と点数を画面に表示するプログラムを作成してみましょう。

また、C 言語基礎編③で学習した内容を応用して、「Record 型構造体の定義を記述するファイル」を別に作成してみましょう（構造体型の宣言・定義はヘッダファイルに記述）。

＜str_Record.h＞

```
// 【1】を参考にして Record 型の定義とマクロ定義を行う

```

＜str_main.c＞

```
#include <stdio.h>
#include “str_Record.h”   //ヘッダファイルインクルード

int main(void)
{
        Record data[N_HUMAN] = {
                { 1, “佐藤”, {88, 76, 69, 79}},
                { 2, “鈴木”, {100, 6, 65, 72}},
                { 3, “高橋”, {5, 100, 19, 99}},
                { 4, “田中”, {98, 98, 96, 92}},
                { 5, “伊藤”, {94, 76, 75, 80}}
        };
        int i, j;
        //各生徒の合計点を格納する配列
        int sum_result[          ] = {};
```

```
        //各生徒のテストの合計点を計算し、sum_result に格納
        for( i = 0; i <          ; i++){
                for( j = 0; j <        ; j++){
                                        
                }
        }

        //合計点が最も高い生徒を探し、その生徒の名前と点数を表示
        int max_result = 0;   //最高点を格納する変数
        int max_ind;   //最高点の生徒の要素番号を格納する変数
        for( i = 0; i <          ; i++){
                if(                          ){
                                                
                                                
                }
        }
        printf( “最高得点は%s さんで、%d 点です¥n”,                          );

        return 0;
}
```

＜実行結果イメージ＞

```
最高得点は田中さんで、384点です
```

3. 参考文献

[1].『やさしい C』，高橋麻奈，ソフトバンクパブリッシング株式会社, 2003 年

[2].『明解 C 言語　入門編』，柴田望洋，ソフトバンククリエイティブ株式会社, 2008 年

[3].『【C 言語】typedef と#define の違い』,（https://hotnews8.net/programming/tricky-code/language-specification03）

4. 演習問題解答例

4.1.　1 章演習問題

【1】

以下のプログラム「pointer_test.c」を実行すると、実行結果がどのようになるか答えてみましょう。

＜pointer_var.c＞

```
#include <stdio.h>

int main(void)
{
	int num = 10;
	int *ptr_num = &num;

	if( num == ptr_num){
		printf("num = ptr_num¥n");
	}
	if( num == *ptr_num){
		printf("num = *ptr_num¥n");
	}
	if( &num == ptr_num){
		printf("&num = ptr_num¥n");
	}
	if( &num == *ptr_num){
		printf("&num = *ptr_num¥n");
	}

	return 0;
}
```

＜実行結果＞

```
num = *ptr_num
&num = ptr_num
```

【2】

以下の条件を満たすプログラム「pointer_func.c」を作成するために、網掛け部分を埋めてみましょう。

・2 つの引数を取り、それぞれの引数に対して以下のような操作を加える関数 add_times を作成する

i） 1 つ目の引数に、2 つ目の引数の値を加算して返す

ii） 2 つ目の引数に、1 つ目の引数の値を乗算して返す

・キーボードから任意の 2 つの値を入力し、関数 add_times の実引数に指定する

・関数 add_times の実行結果を画面に表示する

＜pointer_func.c＞

```
#include <stdio.h>

void add_times( int *input1, int *input2){   //ポインタを引数に取る
        int tmp = *input1;   //値の参照時は間接参照演算子を付ける
        (*input1) += *input2;
        (*input2) *= tmp;
}

int main(void){
        int input1, input2;
        printf( “入力値 1 : ”);
        scanf( “%d”, &input1);
        printf( “入力値 2 : ”);
        scanf( “%d”, &input2);
        //アドレス演算子を付けて参照渡し
        add_times( &input1, &input2);
        printf( “入力値 1 +  入力値 2 = %d¥n”, input1);
        printf( “入力値 1 *  入力値 2 = %d¥n”, input2);
        return 0;
}
```

【3】

以下の条件を満たすプログラム「pointer_list.c」を作成するために、網掛け部分を埋めてみましょう。

・引数として整数の配列を受け取り、要素を値の大きい順に入れ替える関数 sort_list を作成する

・main 関数で、要素の入れ替え前と入れ替え後の配列の値をそれぞれ表示する

＜pointer_list.c＞

```
#include <stdio.h>

void sort_list( int list[ ], int num){
        int i, j, tmp;
        for( i = 0; i < num; i++){
                for( j = i+1; j < num; j++){
                        if( list[i] < list[j]){
                                tmp = list[i];
                                list[i] = list[j];
                                list[j] = tmp;
                        }   //ほかの要素と比較して大きい値を左側に詰めていく
                }
        }
}

int main(void)
{
        int list[5] = { 23, 45, 11, 89, 32};
        int i;
        for( i = 0; i < 5; i++){
                printf( "%d    ", list[i]);
        }
        printf( "¥n↓降順ソート¥n");
        sort_list( list, 5);   //配列を参照渡し
        for( i = 0; i < 5; i++){
                printf( "%d    ", list[i]);
        }
        printf ( "¥n");

        return 0;
}
```

4.2.　2 章演習問題

【1】

以下に示すような、「番号」、「氏名」、「4 回分のテストの点数」をメンバに持つ構造体を作成し、それぞれの値を表示するプログラム str_studentlist.c の網掛け部分を埋めてみましょう。

＜実行結果例＞

番号	氏名	第 1 回	第 2 回	第 3 回	第 4 回
1	佐藤	88	76	69	79
2	鈴木	100	6	65	72
3	高橋	5	100	19	99
4	田中	98	98	96	92
5	伊藤	94	76	75	80

＜str_studentlist.c＞

```
#include <stdio.h>

#define N_TEST 4
#define N_HUMAN 5

typedef struct GradeRecord{      //Record 型の構造体の定義
        int number ;
        char name[50] ;
        int result[N_TEST] ;
}Record;

int main(void)
{
        Record data[N_HUMAN] = {      //生徒の情報を Record 型変数 data に格納
                { 1, “佐藤”, { 88, 76, 69, 79}},
                { 2, “鈴木”, { 100, 6, 65, 72}},
                { 3, “高橋”, { 5, 100, 19, 99}},
                { 4, “田中”, { 98, 98, 96, 92}},
                { 5, “伊藤”, { 94, 76, 75, 80}}
        };
```

```
	//一番上の行を表示
	printf( “番号　氏名　　”);
	int i, j;
	for( i = 0; i < N_TEST; i++){
		printf( “第%d 回 ”, i+1);
	}
	printf( “¥n”);

	//生徒の情報を表示
	for( i = 0; i < N_HUMAN; i++){
		printf( “%3d　　”, data[i].number );
		printf( “%-4s　”, data[i].name );
		for( j = 0; j < N_TEST ; j++){
			printf( “%6d ”, data[i].result[j] );
		}
		printf( “¥n”);
	}

	return 0;
}
```

【2】

【1】で作成した Record 型の配列 data から、各生徒のテストの合計点を求め、その生徒の名前と点数を画面に表示するプログラムを作成してみましょう。

また、C 言語基礎編③で学習した内容を応用して、「Record 型構造体の定義を記述するファイル」を別に作成してみましょう（構造体型の宣言・定義はヘッダファイルに記述）。

＜str_Record.h＞

```
// 【1】を参考にして Record 型の定義とマクロ定義を行う

#define N_TEST 4
#define N_HUMAN 5

typedef struct GradeRecord{
        int number;
        char name[50];
        int result[N_TEST];
}Record;
```

＜str_main.c＞

```
#include <stdio.h>
#include “str_Record.h”   //ヘッダファイルインクルード

int main(void)
{
        Record data[N_HUMAN] = {
                { 1, “佐藤”, {88, 76, 69, 79}},
                { 2, “鈴木”, {100, 6, 65, 72}},
                { 3, “高橋”, {5, 100, 19, 99}},
                { 4, “田中”, {98, 98, 96, 92}},
                { 5, “伊藤”, {94, 76, 75, 80}}
        };
        int i, j;
        //各生徒の合計点を格納する配列
        int sum_result[N_HUMAN] = {};
```

```
	//各生徒のテストの合計点を計算し、sum_result に格納
	for( i = 0; i < N_HUMAN; i++){
		for( j = 0; j < N_TEST; j++){
			sum_result[i] += data[i].result[j];
		}
	}

	//合計点が最も高い生徒を探し、その生徒の名前と点数を表示
	int max_result = 0;	//最高点を格納する変数
	int max_ind;	//最高点の生徒の要素番号を格納する変数
	for( i = 0; i < N_HUMAN; i++){
		if( max_result < sum_result[i]){
			max_result = sum_result[i];
			max_ind = i;
		}
	}
	printf( “最高得点は%s さんで、%d 点です¥n”, data[max_ind].name, max_result);

	return 0;
}
```